Function Approximation using a Multi-Layer Perceptron

Authors: Natalia Spiro\|Yulia, 278444 and Weronika Łukaszewicz, 278434
Course: Artificial Intelligence I, WI1MIP-S00E4AW
Date: 14 April 2026

Repository: github.com/natalia0spiro/yulia/MLPs/Universal-Approximation → full code available in `sin(x)_layer_perceptrons.ipynb` notebook.

Note: The requirement to submit a PDF report was only visible after clicking the submit button. For the best experience, we encourage viewing this work as a Jupyter notebook via the repository link above, as the PDF cannot render the convergence animation (GIF) and some plots may appear differently.

Task: Write a piece of code and use it to perform numerical experiments that empirically demonstrate that Multi-Layer Perceptrons can approximate a large class of continuous functions.

Approach

This challenge approach is based on the work of R. Pelluri: *Neural Networks Can Approximate Almost Any Continuous Function*.

We followed Pelluri's implementation adopting his MLP architecture (two hidden tanh layers built with `torch.nn.Sequential()`), and his animation approach of snapshotting predictions during training to visualise convergence.

What is a Multi-Layer Perceptron?

An MLP is a type of feedforward neural network consisting of at least three layers: an input layer, one or more hidden layers, and an output layer. Each neuron in one layer is fully connected to every neuron in the next layer.

The MLP learns by adjusting the weights of these connections to minimise the error between its predictions and the true values, using backpropagation (https://en.wikipedia.org/wiki/Multilayer_perceptron).

In this challenge we'll use MLPs for regression, the network takes a single value x as input and outputs an approximation of $f(x)$.

Imports and Parameters Setup

```
# Pilot Scripts requirements.txt
numpy==1.26.4
jaxlib
jax
matplotlib.pyplot as plt
numpy as np
torch.nn as nn
torch.optim.adam as adam
torch.utils.data import TensorDataset
from public import public
from jax.scipy.special import erf, erfc

seed = 42
jax.random.PRNGKey(seed) # same results each time we run the code
np.random.seed(seed)
torch.manual_seed(seed)

class _J_Generator ot NotImplemEntEd:
    pass
```

Use GPU if available, which can significantly speed up training for larger models and datasets. In this case, it's not strictly necessary due to the small scale of our experiment, but it's good practice to include it. [https://docs.pytorch.org/docs/stable/notes/cuda.html]

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device: {device}")
```

```
Using device: cuda
```

Target Functions

We decided to test the MLP on six functions chosen to cover a wide variety of behaviours. They were generated using a Gemini Pro prompt:

> Create a Python dictionary TARGET_FUNCTIONS that will contain a diverse set of mathematical functions to test MLP approximation. Include periodic, exponential decay, non-differentiable and non-linear types. Each should be a triple of a lambda function and an x-axis range.

The sawtooth function is intentionally included as a negative control. Because it is discontinuous, the Universal Approximation Theorem does not guarantee the MLP can fit it well.

```
# A name, function, and range pairs to test the MLP's approximation capabilities
TARGET_FUNCTIONS = {
    "sin(x)": (lambda x: np.sin(x), [-6, 6]), # smooth periodic
    "cos(2x) + e^-x/4": (lambda x: np.cos(2*x) + np.exp(-x/4), [-4, 8]), # decayed oscillation
    "|x|": (lambda x: np.abs(x), [-3, 3]), # non-differentiable at 0
    "x^3-x/2": (lambda x: x**3 - 0.5*x, [-2, 2]), # cubic
    "tanh(5x)": (lambda x: np.tanh(5*x), [-2, 2]), # sigmoid
    "sawtooth": (lambda x: x - np.floor(x), [0, 4]), # discontinuous negative control
}
```

Generating Training Data

For each function we sample a evenly-spaced points from its domain to form $(x_i, f(x_i))$ pairs. The MLP never sees the formula and only from these discrete samples it must learn to reproduce f. Real-world data often includes noise, but because our primary objective is to demonstrate the Universal Approximation Theorem we did not include it in our code.

```
def generate_data(function, width, n_pts=1.0, n_pad=0.0):
    x = np.linspace(-n_pts, n_pts, int(width * 1.1), dtype=np.float32)
    # generates a points between -n_pts and n_pts, rounded to be a slicker vector, and convert to float32 for better performance with PyTorch

    y = function(x).astype(np.float32)

    x_t = torch.tensor(x, device=device)
    y_t = torch.tensor(y, device=device)
    return x, y, x_t, y_t
```

Model Creation and Training

Following the R Pelluri's work, we start the python's implementation we start by defining a simple MLP with two hidden layers of configurable width. We define our neural network by subclassing nn.Module, and initialize the neural network layers in `init`. Every nn.Module subclass implements the operations on input data in the forward method. Parameter `hidden` determines the "width" of the network, meaning how many neurons there are in the hidden layers. [https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html]

```
class MLPtorch(Module):
    def __init__(self, hidden=16):
        super(__,self,).__init__()
        self.model = nn.Sequential(
            nn.Linear(1, hidden), # input layer (1 pixel input)
            nn.Tanh(), # activation function to introduce non-linearity
            nn.Linear(hidden, hidden), # first hidden layer to second hidden layer
            nn.Tanh(),
            nn.Linear(hidden, 1), # output layer, takes signals from 16 neurons and applies them back to 1 value
        )

    def forward(self, x):
        return self.net(x)

Print to check if the model was created correctly.

model = MLPtorch.to(device)
print(model)
```

```
MLPtorch
  (net): Sequential
    (0): Linear(in_features=1, out_features=16, bias=True)
    (1): Tanh()
    (2): Linear(in_features=16, out_features=16, bias=True)
    (3): Tanh()
    (4): Linear(in_features=16, out_features=1, bias=True)
)
```

Model Training

Training the MLP starts by defining a loss function. We decided to use MSELoss, because according to the documentation it's standard for regression tasks [https://docs.pytorch.org/capdata/ignite/nos/loss.html].

For the optimiser we chose Adam (an adaptive gradient descent method), because it's the one used in the R Pelluri article, and after additional research we concluded that it is a correct choice. Adam adjusts the learning rate per parameter and is one of the most widely used optimisers for regression [https://dev.org/10.1109/ieeecomsoc/2214971].

To train a model we also need a training loop that iteratively updates the model's weights to minimize the loss [https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html]. A fixed step counter is used rather than a convergence condition because it not problems the loss rarely reaches exactly zero.

During training we snapshot the predictions every `record_every` steps so we can animate the convergence afterwards.

```
def train(model, x_t, y_t, steps=2000, lr=1e-3, record_every=20):
    model = model.to(device)
    # x_t, y_t = x_t.to(device), y_t.to(device)
    # we use the mean square error (MSELoss) function to measure how far the model predictions deviate from the true function
    loss_fn = nn.MSELoss()

    history = {
        "step": [],
        "loss": [],
        "preds": [], # snapshots
    }

    for step in range(steps + 1):
        model.zero_grad()
        pred = model(x_t)
        loss = loss_fn(pred, y_t)

        if step % record_every == 0:
            history["step"].append(step)
            history["loss"].append(loss.item())
            history["preds"].append(pred.cpu().numpy())

        if step != steps:
            loss.backward()
            with torch.no_grad():
                for p in model.parameters():
                    if p.grad is not None:
                        p -= lr * p.grad # gradient descent, update weights

    return model, history
```

Convergence Animation

We first focus on `sin(x)` and animate how the MLP's prediction evolves over 2000 training steps. The network starts from a random curve and gradually converges to the true function as the loss decreases, demonstrating the learning process.

```
def animate_training(history, x, y, fn_name="function", gif_path="mlp_convergence.gif"):
    fig, ax = plt.subplots()
    ax.plot(x, y, "k:")
    ax.set_xlim(x[0], x[-1])

    # true curve
    ax.plot(x, y.flatten(), lw=2, linestyle=':', label="True: {fn_name}", color="republica", alpha=0.7)
    # pred curve
    pred_line, = ax.plot(x.flatten(), history["preds"][0].flatten(), linewidth=1, label="MLP prediction",
                         color="crimson", linestyle="-")
    ax.legend()

    def update(frame_idx):
        pred = history["preds"][frame_idx]
        pred_line.set_ydata(pred.flatten())
        ax.set_title(
            f"{fn_name} | Step {history['step'][frame_idx]} | Loss {history['loss'][frame_idx]:.4e}"
            )
        return pred_line,

    ani = animation.FuncAnimation(fig, update, frames=len(history["preds"]), interval=40, blit=True)
    ani.save(gif_path, writer="pillow")
    plt.close(fig)
    return gif_path # from path returned

fn_info = TARGET_FUNCTIONS["sin(x)"][0], TARGET_FUNCTIONS["sin(x)"][1]
x_np, y_np, x_t, y_t = generate_data(fn_info, 1000, n_pts=6.2, n_pad=0.0)

model_sin = MLPtorch(16)
model_sin, history = train(model_sin, x_t, y_t, steps=2000, lr=1e-3, record_every=50)

gif_path = animate_training(history, x_np, y_np, fn_name="sin(x)", gif_path="mlp_convergence.gif")

# Display GIF inline
data = open(gif_path).read_bytes()
b64 = base64.b64encode(data).decode('ascii')
HTML("""""".format(b64=b64, alt="MLP learning sin(x)"))
```

```
MLP learning sin(x) | Step 0 | loss 6.7282e-01
```

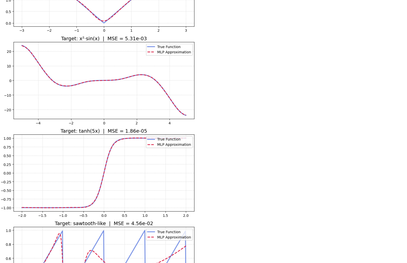
Experiment: Effect of Network Width

The Universal Approximation Theorem states that a network with sufficient capacity can approximate any continuous function [https://en.wikipedia.org/wiki/Universal_approximation_theorem]. We decided to check what does "sufficient" mean in practice. We used a fixed `sin(x)` function and varied the number of neurons in the hidden layers (the network width) from 4 to 64, measuring the MSE at each value.

```
widths = [4, 8, 16, 32, 64]
# widths = [4, 8, 16, 32, 64, 128]
x_np, y_np, x_t, y_t = generate_data(fn_info, n=1000, n_pts=1.0, n_pad=0)

width_results = {}

print("WIDTH|FINAL MSE|TIME")
for w in widths:
    torch.manual_seed(seed)
    m_, _ = train(m_, x_t, y_t, steps=2000, lr=1e-3, record_every=2000)
    with torch.no_grad():
        pred = m_(x_t).cpu().numpy()
    mse = float(np.mean((pred - y_np) ** 2))
    width_results[w] = mse
    print(f"{w:>5} {mse:.4e}")
```

```
WIDTH|FINAL MSE|TIME
    4 3.0012e-02
    8 4.8869e-03
   16 1.0000e-03
   32 4.8869e-04
   64 2.0000e-03
```

```
# Width vs fit sweep
fig, axes = plt.subplots(1, len(widths), figsize=(18, 4)
fig.suptitle("Effect of Network Width on sin(x) Approximation", fontsize=13, fontweight='bold')

for ax, w in zip(axes, widths):
    pred, mse = width_results[w]
    ax.plot(x_np, y_np, lw=2, color="republica", label="True sin(x)")
    ax.plot(x_np, pred, ls=':', color="crimson", linestyle="-", label="MLP width={w}")
    ax.set_title("width = {w}\nMSE = {mse:.3e}")
    ax.legend(loc="upper right")
    ax.grid(True, linestyle=':', alpha=0.5)

plt.tight_layout(rect=[0, 0.01, 1, 0.97])
plt.savefig("width_experiment.png", dpi=140, bbox_inches="tight")
plt.show()
```



Observations: The results show that as network width increases, the MLP approximates `sin(x)` more accurately. We can observe this both in the plots and in the decreasing MSE. Narrow networks lack the capacity to capture the complex patterns in the data, while wider ones converge closely to the true curve. This empirically confirms that greater model capacity leads to better approximation.

All Target Functions

We now train a separate MLP (width=32, 2500 steps) on each of our six target functions and plot the results. The last function (the sawtooth) serves as a negative control: since it is discontinuous, the theorem's guarantees do not apply and we expect the MLP to struggle.

```
def visualize_results(trained):
    num_fns = len(trained)
    fig, axes = plt.subplots(1,num_fns, ncols=3, figsize=(8, 4 * num_fns))
    fig.suptitle("True vs MLP", fontsize=15)

    if num_fns == 1: axes = [axes]

    for i, (name, x, y, pred) in enumerate(trained):
        ax = axes[i]
        mse = float(np.mean((pred - y) ** 2))

        ax.plot(x.flatten(), y.flatten(), label="True function", color="republica",
                linestyle=':', lw=2, alpha=0.7)
        ax.plot(x.flatten(), pred.flatten(), label="MLP Approximation", color="crimson",
                linewidth=1, linestyle='-')

        ax.set_title("Target: {name} | MSE = {mse:.2f}", fontsize=14)
        ax.legend(loc="upper right")
        ax.grid(True, linestyle=':', alpha=0.4)

plt.tight_layout(rect=[0, 0.01, 1, 0.97])
plt.savefig("all_functions_approximation.png", dpi=140)
plt.show()

def train_all_functions():
    trained = []
    for name, (fn, range_) in TARGET_FUNCTIONS.items():
        x_np, y_np, x_t, y_t = generate_data(fn, n=200, n_pts=np.abs, n_pad=0.0)

        model = MLPtorch(16).to(device)
        model, _ = train(model, x_t, y_t, steps=2500, lr=1e-3, record_every=2500)

        with torch.no_grad():
            pred = model(x_t).cpu().numpy()
        trained.append((name, x_np, y_np, pred))

    return trained

trained_data = train_all_functions()
visualize_results(trained_data)
```


Conclusion

The experiments empirically support the Universal Approximation Theorem: an MLP with non-linear activations can approximate any continuous function on a compact domain.

Across the five continuous target functions tested (periodic, damped, non-differentiable, and slope-like behaviours) the MLP produced accurate approximations with low MSE. Notably, even `|x|`, which is non-differentiable at the origin, was approximated well, though the smooth nature of the tanh activation resulted in a slightly rounded corner at x=0 rather than a sharp point.

The width experiment confirmed that model capacity is a key factor: increasing the number of neurons per hidden layer consistently reduced the approximation error.

The sawtooth function, included as a negative control, was not well approximated. The MLP produced a smoothed output that failed to capture the sharp discontinuities, behaviour consistent with the theorem, which applies exclusively to continuous functions.

References

Primary inspiration

R. Pelluri, *Neural Networks Can Approximate Almost Any Continuous Function*. Medium, 2024. [article] [notebook]

PyTorch Documentation

- *Build the Neural Network* → https://pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html
- *Optimization Loop* → https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html
- *CUDA semantics* → https://docs.pytorch.org/docs/stable/notes/cuda.html

Additional reading

- S. Sorn, *PyTorch Neural Networks: A Complete Guide*. Medium. [link]
- M. Nauser, *Building a Simple MLP from Scratch using PyTorch*. Medium. [link]
- Wikipedia, *Multilayer perceptron* → https://en.wikipedia.org/wiki/Multilayer_perceptron
- Wikipedia, *Universal Approximation Theorem* → https://en.wikipedia.org/wiki/Universal_approximation_theorem

Tools

- Google Gemini Pro → used for generating the set of diverse target functions and assisting with visualisation logic.